Part—3

Lisp
Tears of Joy

LISP has been hailed as the world's most powerful programming language. But only the top percentile of programmers use it, because of its cryptic syntax and academic reputation. This is rather unfortunate, since LISP isn't that hard to grasp. If you want to be among the crème de la crème, this series is for you. This is the third article in the series that began in the June 2011 issue of LFY.

Genesis

By now, most of us agree that LISP is the world's greatest programming language. For those of you who still disagree, and think it probably stands for 'Lots of Irritating Superfluous Parentheses', my suggestion would be to read this article with a few bottles of strong beer (for the hacker types) or wine (for the high brow literary elite).

Guy L Steele Jr and Richard P Gabriel, in their paper 'The Evolution of LISP' *(http://www.dreamsongs.com/NewFiles/HOPL2-Uncut.pdf)*, say that the origin of LISP was guided more by institutional rivalry, one-upmanship, and the glee born of technical cleverness that is characteristic of the hacker culture, than by a sober assessment of technical requirements.

How did it all start? Early thoughts about a language that eventually became LISP started in 1956, when John McCarthy attended the Dartmouth Summer Research Project on Artificial Intelligence. Actual implementation began in the fall of 1958. These are excerpts from the essay 'Revenge of the Nerds' by Paul Graham:

"LISP was not really designed to be a programming language, at least not in the sense we mean today, which is something we use to tell a computer what to do. McCarthy did eventually intend to develop a programming language in this sense, but the LISP that we actually ended up with was based on something separate that he did as a theoretical exercise—an effort to define a more convenient alternative to the Turing Machine. As McCarthy said later, "Another way to show that LISP was neater than Turing machines was to write a universal LISP function, and show that it is briefer and more comprehensible than the description of a universal Turing machine. This was the LISP function *eval*, which computes the value of a LISP expression.... Writing *eval* required inventing a notation representing LISP functions as LISP data, and such a notation was devised for the purposes of the paper, with no thought that it would be used to express LISP programs in practice."

Some time in late 1958, Steve Russell, one of McCarthy's graduate students, looked at this definition of *eval* and realised that if he translated it into machine language, the result would be a LISP interpreter.

This was a big surprise at the time. Here is what McCarthy said about it later in an interview: "Steve Russell said, "Look, why don't I program this *eval*," and I said to him, "Ho, ho, you're confusing theory with practice; this *eval* is intended for reading, not for computing." But he went ahead and did it. That is, he